CSCI 441 SOFTWARE ENGINEERING · DEMO 1

Technical Documentation

How to install, run, test and maintain the SuperAsk.AI prototype

Document type	**Technical documentation (developer and system reference)**
Project title	**SuperAsk.AI: answers about Cambodian government services, in Khmer and English, with the official source attached**
Group	**Team AO3**
Members	**Chun Phallamonunhh** **Fong Yean** **Sarin Sereyprophea** **Heng Bunkheang** **Choi Heejae** **Neang Doungkhe**
Date	**9 November 2026**
Repository	**https://github.com/Monunhh/Super-ask**

Describes the code at the root of the repository and the Demo 1 e-archive in `docs/demo-1/`.

Contents

1. Purpose and scope 3

2. Technology stack 3

3. Repository layout and the Demo 1 e-archive 4

4. Installing and running 5

5. Configuration

6. Architecture and request processing 6

7. API routes and the Telegram integration 7

8. Knowledge base and data collection 7

9. Testing and evaluation 8

10. Continuous integration 9

11. Deployment and maintenance 9

12. Known limitations 9

1. Purpose and scope

This document explains how to install, run, understand, test and maintain the SuperAsk.AI prototype shown at Demo 1. It is written for a developer or technical reviewer working from the project repository. The user-facing guide is the separate *User Documentation*.

SuperAsk.AI is a Next.js web application with one shared question-answering engine. Three entry points use that engine: the web chat, a Telegram bot and a Telegram Mini App. The engine answers only from an approved knowledge base, attaches the official source to every answer, and declines when no source supports an answer.

> **Status.** This is a course prototype. The knowledge base is a sample corpus built from public ministry pages; it has not been formally approved by government content stewards. Fees and deadlines in it must not be treated as official.

2. Technology stack

Part	Implementation
Front end	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS 4
Back end / API	Next.js route handlers on the Node.js runtime
Answer engine	Guardrail screen, curated-answer match, hybrid retrieval with rank fusion and re-ranking, confidence gate, generation, claim verification, output moderation, citation assembly (`lib/engine/`, `lib/retrieval/`)
Khmer language	Normalisation, word segmentation and romanisation (`lib/khmer/`)
Model providers	SEA-LION, Anthropic or a self-hosted vLLM server, chosen by `LLM_PROVIDER`; a deterministic extractive mode needs no model at all
Messaging	Telegram Bot API webhook and Telegram Mini App
Data store	Version-controlled JSON files in `data/`; append-only JSONL audit and feedback logs in `var/` (no relational database in Phase 0)
Testing	Vitest with V8 coverage, TypeScript type checking, evaluation and quality-gate scripts
CI and hosting	GitHub Actions on Node.js 22; deployed on Vercel at https://superask.vercel.app

3. Repository layout and the Demo 1 e-archive

Path	Contents
`app/`	Pages (landing, chat, login and register screens) and API routes under `app/api/`
`components/`	React interface: chat composer, conversation, source cards, sidebar, feedback
`lib/engine/`	Three-tier answer engine, guardrails, verification, citations, freshness
`lib/retrieval/`	Search and ranking over the approved knowledge base
`lib/khmer/, lib/lang/`	Khmer processing and language detection
`lib/llm/`	Model provider abstraction and the extractive fallback
`lib/telegram/`	Telegram client, reply formatting, Mini App button
`lib/crawl/, lib/registry/`	Source monitoring crawler and the government-site registry
`lib/log/`	Audit log with personal-data redaction, operator trace
`data/`	Knowledge base, curated answers, glossary, aliases, office directory, evaluation set, source registry
`scripts/`	Evaluation, quality checks, crawler, source-registry builder, Telegram webhook tool
`tests/`	Unit tests and API route integration tests (run by `npm test`)
`docs/`	Course deliverables: proposal, Reports 1 and 2, and the Demo 1 e-archive

Demo 1 e-archive (`docs/demo-1/`)

Folder	Contents
`1_code/`	`README1.txt`: where the code is and how to run it
`2_unit_testing/`	`README2.txt`: where the unit tests are and how to run them
`3_integration_testing/`	Cross-channel integration tests, their Vitest configuration and `README3.txt`
`4_data_collection/`	`README4.txt`: data collection scripts and how to run them
`5_documentation/`	The five PDF documents, including this one
`slides/, documents/`	Sources and build scripts for the slides and for these documents

The application code, the unit tests and the data collection scripts stay at the repository root rather than being copied into `1_code`, `2_unit_testing` and `4_data_collection`. Next.js requires its source folders at the root, the tests import that code by path, and a copy would become a second, out-of-date version. Each of those folders holds a README that points to the real location.

4. Installing and running

Requirements: Node.js 22 or newer (the version CI uses), npm, and an internet connection for the first install. Windows, macOS and Linux all work.

1. Clone the repository and open its root folder:

```
git clone https://github.com/Monunhh/Super-ask.git
cd Super-ask
```

2. Install the dependencies:

```
npm install
```

3. Create a local settings file. macOS and Linux: `cp .env.example .env.local`. Windows PowerShell: `Copy-Item .env.example .env.local`.
4. Start the development server and open http://localhost:3000:

```
npm run dev
```

5. Optional production build: `npm run build`, then `npm start`.

No model credential is needed. Without one, the engine uses its deterministic extractive mode, which quotes the approved source passage directly. Answers from a model-backed deployment are worded differently but use the same sources and gates.

5. Configuration

Settings are environment variables, documented in `.env.example`. Keep real keys in `.env.local` locally and in the host's secret settings in deployment; never commit them, and never expose them to browser code.

Variable	Purpose
`LLM_PROVIDER`	`sealion`, `anthropic`, `vllm` or `extractive`
`SEALION_*, ANTHROPIC_*, VLLM_*, LLM_MODEL`	Keys, base URLs and model names for the chosen provider

Variable	Purpose
`CURATED_MATCH_THRESHOLD`	How close a question must be to serve a curated answer
`RETRIEVAL_MIN_SCORE`	Decline floor: below this retrieval score SuperAsk declines (0.20)
`DIAGNOSTICS_ENABLED`, `ASK_TRACE`	Operator diagnostics and request tracing; off in CI and production
`TELEGRAM_BOT_TOKEN`, `TELEGRAM_WEBHOOK_SECRET`	Telegram bot credentials and the secret that authenticates webhook calls
`TELEGRAM_MINI_APP_URL`, `NEXT_PUBLIC_SITE_URL`	Where the Mini App button and links point

6. Architecture and request processing

All three entry points call one function, `ask()` in `lib/engine/`, so the web chat, the bot and the Mini App always give the same answer to the same question. The engine has three tiers: a human-approved curated answer served verbatim (tier 1), retrieval-augmented generation over the knowledge base (tier 2), and a decline (tier 3). A question passes through these stages in order:

#	Stage	What it does
1	Guardrail screen	Declines emergencies, legal advice, personal cases, predictions, political and off-domain questions before any retrieval or model call
2	Curated match	Serves an approved curated answer verbatim when the question matches one
3	Retrieval	Keyword and Khmer-aware search over `data/kb/`, aliases for query expansion, rank fusion and re-ranking
4	Confidence gate	Declines when the best passage scores below the decline floor, so weak retrieval never reaches generation
5	Generation	Writes the answer from the retrieved passages only, in the language of the question
6	Verification	Checks figures and claims against the sources; suppresses an answer that states something the sources do not
7	Output moderation	Final policy check on the written answer
8	Citation assembly	Attaches ministry, document, effective and last-verified dates, review status and link

Every answer, including declines, is written to an audit record (question, retrieved source IDs, model and prompt version, answer, verification outcome) after personal data such as phone and ID numbers has been redacted.

The knowledge base and the monitored websites are kept apart: the crawler records that an official page changed, but fetched text never becomes answerable automatically. A person must review it and update `data/kb/`.

7. API routes and the Telegram integration

Route	Purpose
`POST /api/ask`	Takes a question (required, at most 1,000 characters) and recent history; returns the answer or decline, citations and confidence. An unexpected error returns a safe decline, never an unsupported answer.
`POST /api/feedback`	Records a report or rating on an answer, with its answer ID and citations, comment redacted
`GET /api/health`	Health check for deployment monitoring
`POST /api/telegram/webhook`	Receives Telegram updates, checks the webhook secret, answers through the same engine and replies in Khmer or English with a Mini App button

To connect a bot, set the token and webhook secret, then register the webhook from a trusted machine:

```
npx tsx scripts/telegram-webhook.ts set https://<host>/api/telegram/webhook
```

8. Knowledge base and data collection

Data	Count	Description
`data/kb/*.json`	45	Source passages from five ministry areas (driving licence, vehicle registration, civil registration, trademarks, consumer finance), each with provenance, URL and dates
`data/curated.json`	5	Human-approved answers served verbatim
`data/glossary.json`	14	Official terms explained in Khmer and English
`data/aliases.json`	37	Everyday words mapped to official terms for search
`data/facilities.json`	2	Office directory entries and emergency numbers (UC-7)

Data	Count	Description
`data/sources.json`	132	Generated registry of monitored government websites (not answerable content)
`data/golden.json`	44	Evaluation set: 34 questions that should be answered, 10 that should be declined

Data collection scripts are described in `docs/demo-1/4_data_collection/README4.txt`. In short: `npm run build-sources` regenerates `data/sources.json` from the registry; `npm run crawl:verify` checks every registered site; `npm run crawl:monitor` detects changed pages. The crawler respects robots.txt, identifies itself, and waits at least 2 seconds between requests to the same host. Do not edit `sources.json` by hand.

9. Testing and evaluation

Command	What it runs
`npm test`	Unit tests and API route integration tests in `tests/`
`npm run test:coverage`	The same, with V8 coverage and enforced minimum thresholds
`npm run test:integration`	Cross-channel integration tests in `docs/demo-1/3_integration_testing/`
`npm run typecheck`	TypeScript type check
`npm run eval`	Retrieval and refusal evaluation on the 44-question set (no model calls)
`npm run check`	Type check, tests, evaluation and the quality gates (verify, curated, scope, rationale)

Results on 9 November 2026

Measure	Result	Where
Unit tests	610 passed, 0 failed	`tests/`
API route integration tests	33 passed, 0 failed	`tests/api/routes.test.ts`
Cross-channel integration tests (web, Telegram)	9 passed, 0 failed	`3_integration_testing/`
Line coverage	89.6%	`npm run test:coverage`
Branch coverage	94.1%	`npm run test:coverage`

Measure	Result	Where
Function coverage	96.0%	`npm run test:coverage`
Right source in the top 8 results (target 85%)	94.1%	`npm run eval`
Right source ranked first	67.6%	`npm run eval`
Out-of-scope questions declined (target 98%)	100%	`npm run eval`

In total 652 automated tests: 610 unit and 42 integration (33 + 9).

10. Continuous integration

GitHub Actions (`.github/workflows/ci.yml`) runs on every push to `main` and every pull request, on Ubuntu with Node.js 22 and no model credential, so no paid model is ever called. The steps are: install with `npm ci`, type check, tests with coverage thresholds, production build, the retrieval and refusal evaluation, and the quality gates. A drop below the coverage floor or the evaluation targets fails the build.

11. Deployment and maintenance

- Deploy by building the project on the host with the environment variables set there, then check `GET /api/health`, the web chat and the Telegram bot.
- After changing the engine, retrieval rules, glossary, aliases or knowledge base, run `npm run check` before merging; CI runs it again.
- To add content, add passages to `data/kb/` with source URL and dates, add evaluation questions to `data/golden.json`, and re-run the evaluation.
- Audit and feedback logs in `var/` are git-ignored; personal data is redacted before it is written.

12. Known limitations

- The knowledge base is a small sample covering five service areas, not yet approved by ministry stewards.
- The login and register screens are prototypes; there are no real user accounts yet.
- There is no reviewer portal yet for changes found by the crawler.
- Ranking puts the right source first 67.6% of the time; improving this is planned before Demo 2.
- Telegram follow-up memory can be lost when the serverless host restarts.
- The office directory (UC-7) has two entries; map links are planned.